BLOCKSEC

Security Audit

Report for axcoin-evm-contracts

Date: July 27, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Report Manifest

Item	Description
Client	AX Coin Bahrain B.S.C (C)
Target	axcoin-evm-contracts

Version History

Version	Date	Description
1.0	July 27, 2026	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-07-28 02:58:05+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository [1] of axcoin-evm-contracts of AX Coin Bahrain B.S.C (C).

The PoRGatedMinter is a permissioned mint gate for the existing Fireblocks ERC20 stablecoin, AXUSD, that routes issuance through the multisig Operations vault. Each mint must come from an approved mint requester and pass a fresh proof-of-reserve check, a request deduplication check, and an optional per-call mint limit before AXUSD can be issued.

Note this audit only focuses on the smart contracts in the following directories/files:

- contracts/PoRGatedMinter.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
axcoin-evm-contracts	`Version 1`	`ede5d2c0536f352894e7345d63bd7ec76997b3fb`
	`Version 2`	`b84b47e994be9c2b07f6a1f511aac25ad189c7f8`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not

[1] https://github.com/AlloyXGroup/axcoin-evm-contracts

guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we have **two** recommendations and **four** notes.

- Recommendation: 2
- Note: 4

ID	Severity	Description	Category	Status
1	-	Add zero-address checks	Recommendation	Fixed
2	-	Add zero-amount validation	Recommendation	Fixed
3	-	Deprecated Chainlink round field	Note	-
4	-	Assumptions on external contracts	Note	-
5	-	Potential centralization risks	Note	-
6	-	Future L2 deployments should account for sequencer availability	Note	-

The details are provided in the following sections.

2.1 Recommendation

2.1.1 Add zero-address checks

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `PoRGatedMinter`, the constructor stores `_token` and `_porFeed` and grants `DEFAULT_ADMIN_ROLE` to `_admin` without validating that these addresses are non-zero. The function `setPorFeed()` also accepts a zero `_feed` address. A zero token or feed address causes invocations of dependent functions to revert, while a zero admin leaves no account with the initial admin role. It is recommended to validate these addresses before storing them or granting the role.

```
71        token = IERC20FMintable(_token);
72        porFeed = IAggregatorV3(_porFeed);
73        maxFeedAge = _maxFeedAge;
74        _grantRole(DEFAULT_ADMIN_ROLE, _admin);
75    }
```

Listing 2.1: contracts/PoRGatedMinter.sol

```
78        porFeed = IAggregatorV3(_feed);
79        maxFeedAge = _maxFeedAge;
80        emit PorFeedUpdated(_feed, _maxFeedAge);
81    }
```

Listing 2.2: contracts/PoRGatedMinter.sol

Suggestion Add zero-address checks for `_token`, `_porFeed`, `_admin`, and `_feed`.

2.1.2 Add zero-amount validation

Status Fixed in `Version 2`

Introduced by `Version 1`

Description The function `mint()` consumes a non-zero request ID (i.e., `ref`) after the reserve checks, even when the amount is zero. If the configured token accepts zero-value mints, invoking the function with a zero amount marks `usedRefs[ref]` as true and emits a `GatedMint` event without increasing the token supply. It is recommended to reject zero amounts before the reference is consumed.

```
120       if (ref == bytes32(0)) revert InvalidRef();
121       if (usedRefs[ref]) revert RefAlreadyUsed(ref);
122       uint256 cap = maxMintAmount;
123       if (cap != 0 && amount > cap) revert ExceedsMintCap(amount, cap);
124
125       uint256 r = reserves();
126       uint256 newSupply = token.totalSupply() + amount;
127       if (newSupply > r) revert ExceedsReserves(newSupply, r);
128
129       usedRefs[ref] = true; // effects before interaction
130       token.mint(to, amount);
131       emit GatedMint(to, amount, r, newSupply, ref);
132   }
```

Listing 2.3: contracts/PoRGatedMinter.sol

Suggestion Add a zero-amount check at the beginning of the function `mint()`.

2.2 Note

2.2.1 Deprecated Chainlink round field

Introduced by `Version 1`

Description The interface `IAggregatorV3` retains `answeredInRound` in the return values of the function `latestRoundData()` to match the ABI of Chainlink's `AggregatorV3Interface`. Chainlink marks this field as deprecated, and the function `reserves()` does not use it. The field is therefore an interface compatibility requirement rather than a separate round validity check.

2.2.2 Assumptions on external contracts

Introduced by `Version 1`

Description The security model of the contract `PoRGatedMinter` relies on several external assumptions. The configured `token` is assumed to be a standard ERC20-compatible mintable token whose `totalSupply()`, `decimals()`, and `mint()` behavior are reliable and non-malicious. The configured PoR feed is assumed to follow the expected Chainlink-style interface and to provide correct, timely reserve data. Direct minting from the Operations vault is assumed to be revoked, so successful minting must pass through `PoRGatedMinter` rather than circumventing the reserve gate.

2.2.3 Potential centralization risks

Introduced by Version 1

Description In this project, several privileged roles (e.g., DEFAULT_ADMIN_ROLE and MINT_REQUESTER_ROLE) can conduct sensitive operations, which introduces potential centralization risks. For example, DEFAULT_ADMIN_ROLE can update the PoR feed, change the maximum mint amount, and manage role membership, while MINT_REQUESTER_ROLE can request mints through the function mint() subject to the configured PoR gate and mint cap. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.2.4 Future L2 deployments should account for sequencer availability

Introduced by Version 1

Description AXUSD is initially issued on Ethereum. The project may extend issuance to additional blockchain networks in the future. If AXUSD is deployed to an L2 network in the future, the deployment and integration design should account for L2-specific infrastructure assumptions, including sequencer availability checks where protocol operations depend on sequenced state, oracle updates, bridge state, or other time-sensitive L2 data.


BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS